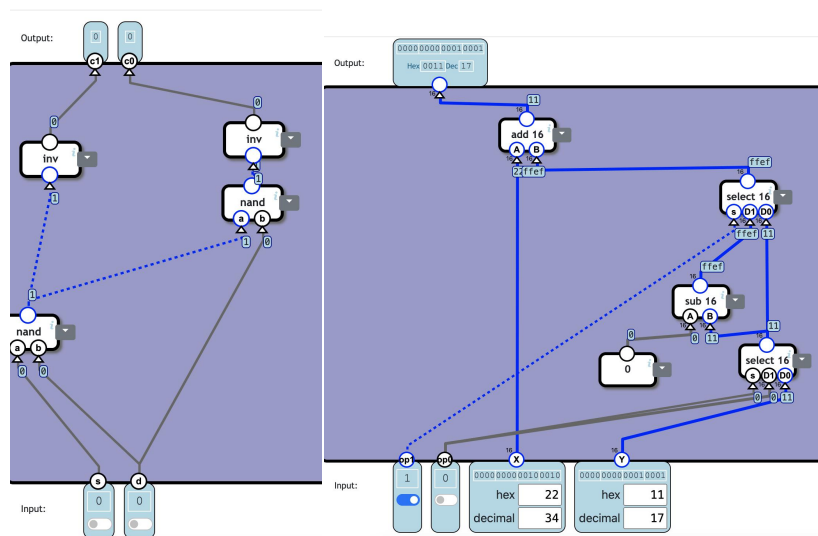


一、完成 Switching 和 Arithmetic Logic Unit



耗时 10 分钟不到，并且都是最佳解。对于第一题，可以观察到，当 $s=0$ 时 $c1=0, c0=d$ ，当 $s=1$ 时， $c1=d, c0=0$ ，根据这个规律来构建元件。然后对于第二题，我们可以观察到当 $op1=0$ 时是加法， $=1$ 时是减法，然后 $op2$ 决定是 Y 还是 1 ，根据这个规律来构建元件。收获：先观察需求，根据需求看看如何化简，再来搭建元件会更加简单。

二、简单运算模块

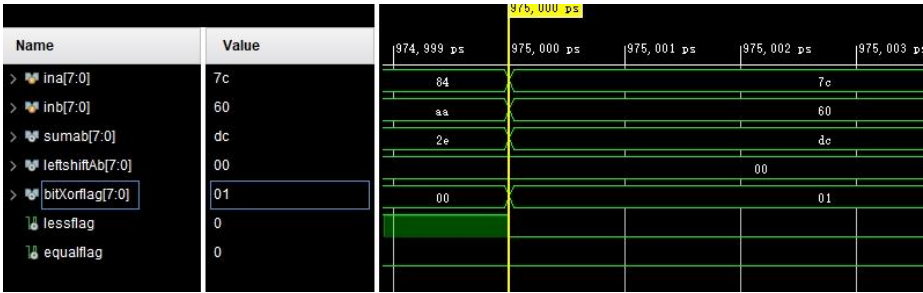
思路比较简单，其实就是课上学到的各种运算符的简单运用，在仿真的时候每次更新随机值放进器件里面跑。如图所示

```
module ysmod(  
    input [7:0] ina,  
    input [7:0] inb,  
    output [7:0] sumab, //这里进位不考虑，直接忽略  
    output [7:0] leftshiftA,  
    output lessflag,  
    output equalflag,  
    output [7:0] bitXorflag  
);  
    assign sumab = ina + inb;  
    assign leftshiftA = ina<<inb ;  
    assign lessflag = (ina < inb ? 1 : 0);  
    assign equalflag = (ina == inb ? 1 : 0);  
    assign bitXorflag = ^ina;  
endmodule
```

元件代码

```
module test1(
);
  reg [7:0] ina;
  reg [7:0] inb;
  always #5
  begin
    assign ina = $random % 9'b1_0000_0000;
    assign inb= $random % 9'b1_0000_0000;
  end
  wire [7:0] sumab;
  wire [7:0] leftshiftAb;
  wire [7:0] bitXorflag;
  wire lessflag;
  wire equalflag;
  ysmod ys(ina, inb, sumab, leftshiftAb, lessflag, equalflag, bitXorflag);
endmodule
```

仿真代码



仿真结果

三、实验心得

本次课程的内容比较简单，主要是介绍了各种运算符，其中全等于和缩减运算符是比较新的，区别于 c++ 的，然后在实验部分也主要是对一些运算符的简单使用，在实验过程中也巩固了 vivado 的基础使用流程。